

Real-Time Musical Sequencing Through Multi-Layer Perceptron Hand Gesture Recognition

Iris Randolph
Harvey Mudd College
Claremont, CA
irandolph@hmc.edu

Sayema Lubis
Harvey Mudd College
Claremont, CA
slubis@hmc.edu

1. Introduction

Modern computer vision (CV) technology has increasingly enabled new natural, intuitive forms of human-computer interaction. One example is how we’ve been able to move beyond traditional input devices to gesture recognition. One particularly compelling example comes from Imogen Heap, whose work with gesture-controlled musical systems demonstrates how expressive hand movements can be translated into sound in real time [6]. Such systems suggest new possibilities for creative expression, accessibility, and multimodal interaction, especially in domains like music where physical interfaces can otherwise present barriers to entry.

Motivated by these ideas, this project explores how core techniques in computer vision can be applied to build a simplified gesture-to-sound interface. Rather than replicating the full complexity of professional gesture-based music systems, we focus on a constrained but meaningful task: recognizing a small set of discrete hand gestures and mapping them to musical outputs. Specifically, we implement a real-time system capable of classifying six hand gestures (fingers counting from zero to five) and translating these classifications into musical tones, played in real-time. In doing so, the project serves both as a practical application of CV methods and as a proof of concept for accessible, gesture-driven interaction.

2. Method

2.1. Datasets

A major component of this project involved identifying and refining an appropriate dataset for training a reliable hand-gesture classifier. Because our system depends on accurately distinguishing between six gesture classes corresponding to the numbers zero through five, the diversity and realism of the training data played a critical role in overall performance. Throughout development, we experimented with three different datasets, each of which influenced the direction and evolution of the project.

Our initial experiments used a grayscale hand dataset containing approximately 18,000 images (see Figure 1). This

dataset provided several advantages during early prototyping: it was relatively large, included both left and right hands, and offered clear, well-centered examples of each gesture class. As a result, it allowed us to quickly validate our landmark extraction and classification pipeline. However, despite its size, the dataset exhibited a major limitation: nearly all images depicted hands in highly standardized poses, with palms facing directly toward the camera and fingers fully splayed apart. While this consistency simplified classification, it also introduced significant overfitting concerns. The model became overly dependent on these idealized poses and struggled to generalize to more natural hand configurations encountered in live testing.

To fix this issue, we experimented with a second dataset composed of RGB hand images (see Figure 2). These images had more natural variations in pose, with hands facing away from the camera and less rigid finger positions. However, the dataset was extremely limited in scale, with only 18 images per class. Even though the variety it offered was enticing, it was unable to solve the overfitting issue due to this small scale.

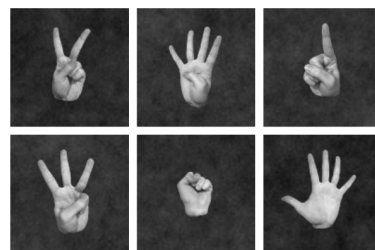


Figure 1. Sample images from the Fingers Dataset [3]. The dataset contains grayscale images of hand gestures corresponding to finger counts from 0–5 under varying hand poses and lighting conditions. These images were used to improve gesture diversity and robustness during classifier training.

The most significant advancement in the project came when we discovered the Hand Gesture Recognition Image Dataset (HaGRID) [4]. HaGRID provided the scale and diversity needed for effective real-world gesture recognition and so ultimately became the foundation for our training pipeline. The dataset contains images from over 65,000 unique individuals and environments, making it much more



Figure 2. Sample images from the Counting Fingers Dataset [5] used for training and evaluation. The dataset consists of RGB hand gesture images representing finger counts from 0–5. MediaPipe hand landmark detection was applied to each image to extract 21 hand landmarks as input features for the MLP classifier.

representative of real-world data than the previous datasets. Unlike the grayscale dataset we initially used, HaGRID includes substantial variation in background scenes, hand orientation, lighting conditions, skin tone, and distance from the camera. This degree of variation allows for the resulting model to generalize much more effectively.

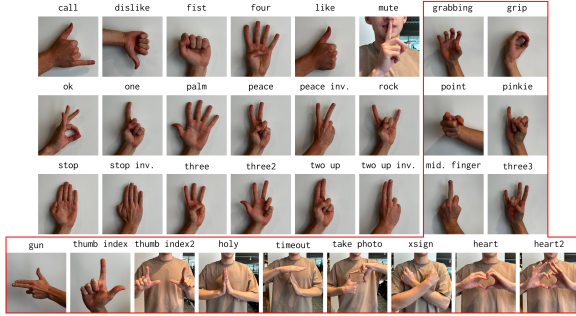


Figure 3. The hand gesture classes of the HaGRID dataset.

Another major advantage of HaGRID is its inclusion of natural pose variation for the counting numbers. Rather than containing a single rigid version of each number gesture, the dataset contains multiple valid hand configurations with the same numerical value. For example, HaGRID contains three different poses denoting the number three, labeled ‘three’, ‘three2’, and ‘three3’, respectively, each using a different combination of fingers 3. This variability aligned much more closely with the goals of our project, since real users rarely perform gestures the same way.

However, integrating HaGRID also introduced several preprocessing challenges. The dataset was designed for broad gesture recognition tasks and therefore included many gesture categories beyond the six classes required for our application. Additionally the gestures like ‘three2’ have multiple variants that needed to be consolidated into unified classes. As part of our preprocessing pipeline, we filtered out irrelevant gesture categories and merged semantically equivalent labels into the six target classes corresponding

to numbers zero through five. Once this restructuring was complete, HaGRID provided an ideal source of training data for extracting MediaPipe landmarks and training our gesture classifier.

The progression through these three datasets reflects an important aspect of the development process: the quality and diversity of training data often have a greater impact on system performance than the classification architecture itself. Early datasets enabled rapid experimentation, but only the large-scale variability present in HaGRID allowed the system to achieve the robustness necessary for real-time gesture recognition in unconstrained environments.

2.2. MediaPipe

One of the key challenges of this project is that of reliably interpreting hand gestures from visual input. Directly processing raw image data introduces a lot of complexity, with variations in lighting, background, skin tone, camera quality, and more. To address these challenges we utilize a structured representation of a hand’s pose, extracted from raw image data. The framework we use to accomplish this is MediaPipe, an open-source platform developed by Google for building real-time CV pipelines.

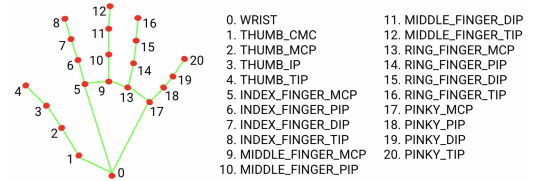


Figure 4. The 21 hand landmarks utilized by MediaPipe’s hand landmarker.

MediaPipe provides a robust real-time hand tracking tool [2] that detects and localizes 21 key landmark points on each hand in an image frame, corresponding to features like finger joints, fingertips, and the wrist (see Figure 4). This landmark-based representation makes for a compact and meaningful encoding of hand geometry, making it very well suited for parsing hand gestures. By abstracting away raw image details, the system becomes more efficient and more resilient to environmental variability.

To incorporate MediaPipe, our project separates the gesture classification problem into two stages: (1) hand detection and landmark extraction, handled by MediaPipe, and (2) gesture classification, performed by a custom-trained multi-layer perceptron model. During training, we apply the MediaPipe hand tracker to a dataset of labeled images to extract feature vectors corresponding to the 21 landmarks [2]. These features are then used to train the classifier to distinguish between the 6 target gestures. At inference time, the system processes a live video stream frame by frame, extracting landmarks in real time and feeding them into the trained classifier to predict the current hand gesture.

The final stage of the pipeline maps these gesture predictions to musical tones that are overlaid on the live video feed. This end-to-end system demonstrates how modular

CV components can be combined to create an interactive application: MediaPipe handles low-level perception, while the classification and audio-mapping layers enable higher-level interpretation and user interaction.

2.3. Multi-Layer Peceptron (MLP)

A mutli-layer perception or in other words, a one- to two-layer dense neural network was selected as the final gesture classification model [1]. The choice of MLP was guided by the two-stage structure of our pipeline, which utilizes MediaPipe as an efficient feature extractor. While our initial experiments involved a Convolutional Neural Network (CNN) trained directly on raw RGB image data, this approach proved ineffective because the problem was simplified by the pre-trained solution that already has robust landmark extraction. Since the input to the classifier is a structured set of geometric features rather than a raw image, the computationally simpler MLP is well-suited to learn the specific geometric relationships that define our six simple gesture classes.

The classification stage operates on a 72-feature input vector constructed from the extracted MediaPipe landmarks. The base of this vector consists of the 21 key landmark points detected by MediaPipe, each defined by three coordinates (x, y, z) yielding 63 feature values. To create more unique distinctions between gestures and capture meaningful geometry, an additional 9 features were appended. These include the distances between each finger and the write, as well as the y-distance of non-thumb fingertips relative to their lowest joint, which serves to effectively measure the number of non-thumb fingers that are extended. Crucially, all values are normalized by a wrist-based value. This ensures that the hand sizes and distances from the camera will not matter, allowing for scale-invariant prediction.

The final architecture consists of a dense network designed to map the 72 input features to the six target gesture classes. The output layer uses a Softmax activation function to generate a probability distribution over the classes, and the model is trained using the cross-entropy loss function, a standard choice for multi-class classification tasks. A diagram of this architecture is shown in Figure 5.

It is important to note that, due to time constraints, we were not able to train and evaluate the model using the larger HaGRID dataset. However, the current implementation establishes a strong foundation for future experimentation and development which will be expanded on in Section 5 and 6.

The code to create and train our model can be seen in this repository.

3. Model Evaluation

To better understand the effectiveness of the hand gesture classifier, additional evaluation methods were implemented to analyze the model’s behavior on both static images and continuous video input. Since the goal of this project was to develop a real-time hand gesture recognition system, it was

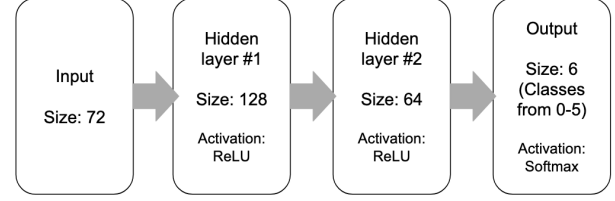


Figure 5. Simplified architectural diagram of the gesture classification pipeline. The input is a 72-feature vector derived from 21 MediaPipe landmarks and 9 hand-engineered features, which is processed by the Multi-Layer Perceptron (MLP) and mapped to the six target gesture classes via a Softmax output layer.

important to evaluate the model beyond simple prediction outputs on isolated images.

Two evaluation approaches were used. First, a confusion matrix was generated to analyze how frequently each gesture class was correctly identified or misclassified. This helped identify patterns of confusion between similar hand poses and provided insight into the limitations caused by the small training dataset.

Second, a ground-truth video labeling pipeline was developed to evaluate the classifier on sequential video frames. By manually labeling video frames and comparing them against model predictions on a frame-by-frame basis, we were able to assess how the classifier performed in a more realistic setting involving continuous hand movement and gesture transitions.

Together, these evaluations provided a more comprehensive understanding of the model’s strengths, weaknesses, and suitability for the real-time implementation.

3.1. Confusion Matrix Analysis

Figure 6 shows the confusion matrix obtained from evaluating the MLP classifier trained on the RGB hand gesture dataset. The matrix compares the ground-truth labels against the predicted labels for gesture classes 0–5.

The model used for this experiment was trained using the RGB dataset collected from the Counting Fingers Dataset (Figure 2). Due to the limited size of the dataset, only 18 images per class were available for training and testing. As a result, the confusion matrix contains relatively low numerical counts across all classes.

Despite the small dataset size, the confusion matrix demonstrates that the model was still capable of correctly identifying several gesture classes. However, some confusion can be observed between visually similar hand poses, particularly between adjacent finger-count classes. This is expected given the limited amount of training data and the variability in hand orientation, lighting conditions, and finger positioning.

The results suggest that increasing the dataset size and including a wider range of hand poses and transitional

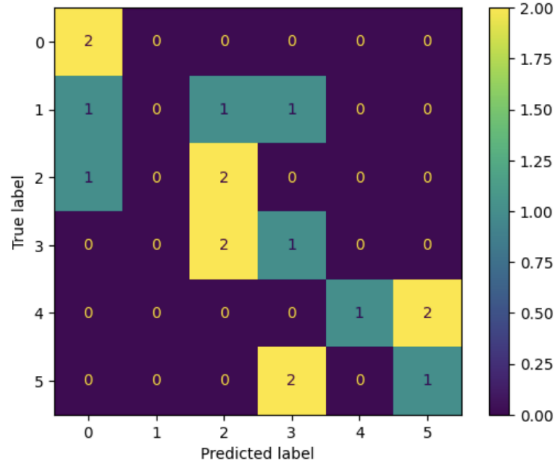


Figure 6. Confusion matrix for the MLP hand gesture trained on the RGB finger-count dataset. Rows represent the ground-truth labels, while columns represent the predicted labels. The relatively low counts are due to the limited dataset size of 18 images per gesture class.

gestures would likely improve classification accuracy and reduce inter-class confusion. This ultimately motivates the switch to the HaGRID dataset more.

3.2. Ground-Truth Labeling Using Video Input

To do the ground-truth labeling, we collected our own video input and manually assigned ground-truth labels corresponding to the number of detected fingers (classes 0-5). These labels were compared against the model predictions on a frame-by-frame basis.

Using this labeled video data, we developed a processing pipeline capable of generating per-frame gesture predictions from video streams. This pipeline establishes the foundation for the system’s real-time hand gesture recognition configuration.

A total of 327 video frames were labeled and evaluated. Despite the relatively small dataset and the presence of transitional hand movements, the classifier achieved an accuracy of approximately 65%, which was higher than initially expected given the variability in real-world hand poses and lighting conditions.

Figure 7 shows example frames used during the ground-truth labeling process. Frames containing clear and stable gestures were assigned labels corresponding to the displayed finger count, in this example, the gesture is labeled ‘2.’ Transitional frames occurring while the hand was moving between gestures were labeled as `None`, since these intermediate poses were not represented in the training dataset and would therefore not provide a fair comparison against the model predictions.

4. Real-Time Implementation

After training the MLP classifier offline, the trained model was deployed into a real-time gesture recognition



Figure 7. Example frames used for ground-truth labeling of video input. The stable hand gestures were assigned labels according to finger count classes 0-5, while transitional frames were labeled “None” to avoid unfair evaluation on poses not represented in the training dataset.

and audio generation pipeline. The offline stage consisted of extracting hand landmark features from labelled gesture datasets and training the classifier to predict gesture classes corresponding to finger counts from 0-5. Once trained, the model was saved and later loaded during runtime for live inference.

Figure 8 illustrates the overall system workflow. In the offline stage, hand landmark features were extracted from the training images and used to train the MLP classifier. The trained model was then deployed to the real-time system.

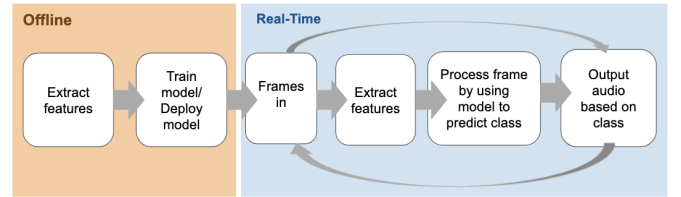


Figure 8. Offline and real-time processing pipeline for the gesture-controlled audio system. The offline stage consists of feature extraction and MLP classifier training. The trained MLP is then deployed into a real-time loop where webcam frames are processed continuously to generate gesture predictions and corresponding chord outputs.

During runtime, webcam frames were continuously streamed into the system using OpenCV. Each incoming frame was processed using MediaPipe to detect hand landmarks and extract feature vectors corresponding to the detected hand pose. These features were then passed into the deployed classifier to predict the gesture class for each frame.

The predicted gesture class was mapped to a corresponding musical chord output. Specifically, gesture classes 1-5 were mapped to C major, D minor, A minor, F major and G minor chords, respectively. Audio playback was implemented using PyAudio, allowing the generated chord to continue playing for as long as the gesture remained visible in the webcam feed [7].

Because predictions were generated independently for every incoming frame, the system continuously updated both the gesture prediction and audio output in real time. This frame-by-frame processing loop established the foundation for an interactive gesture-controlled musical interface.

The full implementation of the real-time system, including the webcam processing pipeline and audio playback functionality, is available in this repository. The system is designed to be modular, allowing users to load and test their own trained gesture classification models for real-time prediction and interaction.

5. Conclusion

This project explored the use of computer vision techniques to create a real-time gesture-controlled music interface, inspired by musician Imogen Heap. By combining hand landmark detection through MediaPipe with a custom gesture classification pipeline, we developed a tool capable of recognizing six hand gestures counting 0 through 5 and mapping them to musical outputs in real time. The project demonstrates how modern CV frameworks can be leveraged to build intuitive interactive applications.

A major takeaway from the development process was the importance of dataset quality and diversity for machine learning systems. Early experimenting with smaller and more constrained datasets caused overfitting issues, particularly when gesture poses were overly standardized. The more expansive nature of the HaGRID dataset, by comparison, exemplifies the large-scale variability needed to create a robust gesture classifier. This progression highlights how model performance is based not just on architecture, but on the representativeness of training data.

Furthermore, the modular pipeline allows different datasets, feature extraction methods, and model architectures to be integrated and evaluated with minimal modifications. Expanded more on Section 6, future work can focus on experimenting with larger and more diverse datasets, such as HaGRID, in order to determine the most optimal configuration for real-time gesture recognition and interactive audio generation.

Beyond the technical, this project reinforced broader ideas about accessibility and alternative forms of human-computer interaction. A gesture-based system provides for those who do not have access to musical instruments or have a music background the power to create music at the palm of their hands.

6. Future Work

Although the final system successfully demonstrated real-time gesture recognition and musical interaction, several important extensions remain for future development. The most immediate area for improvement involves the continued integration and utilization of the HaGRID dataset. While we successfully adapted portions of HaGRID to our pipeline, time constraints prevented us from fully retraining and optimizing the classifier using the complete processed dataset. Because HaGRID contains substantially greater diversity in pose, lighting, background, and user variation than our earlier datasets, fully integrating it would likely improve both classification accuracy and robustness during live inference.

A major extension we would have liked to make would be expanding beyond the current six gesture classes. Our implementation was intentionally scoped to only recognize gestures for the numbers 0 through 5 to allow us to focus on getting the end-to-end pipeline done by the deadline. But the underlying architecture is capable of supporting many more classes, which would make for a more complete-feeling musical architecture. We could even map additional gestures to other control mechanisms such as volume or tone modulation.

Overall, while the current project demonstrates the feasibility of a real-time CV-based musical interface, it also represents a foundation for a much more robust and expressive potential extension. The full integration of the expansive HaGRID dataset and expansion to a more sophisticated vocabulary of gestures would greatly increase the creative potential this project offers to users.

Acknowledgments

We would like to sincerely thank Professor Calden Wloka for his support and guidance throughout this project. His instruction in computer vision provided much of the foundational knowledge that made this work possible, and his feedback and encouragement were a driving force during the development of the real-time gesture recognition system.

References

- [1] DataCamp. Multilayer perceptrons in machine learning. <https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning>, 2024. Accessed: 2026-05-07.
- [2] Google AI for Developers. Mediapipe hand landmarker. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2025. Accessed: 2026-05-07.
- [3] Piyush Joshi. Counting fingers dataset. <https://www.kaggle.com/datasets/piyushjoshi01/counting-fingers-dataset>, 2023. Accessed: 2026-05-07.
- [4] Alexander Kapitanov. Hagrid – hand gesture recognition image dataset. <https://github.com/hukenovs/hagrid>, 2026. GitHub repository, accessed 2026-05-07.
- [5] Pavel Koryakin. Fingers dataset. <https://www.kaggle.com/datasets/koryakin/fingers>, 2017. Accessed: 2026-05-07.
- [6] Mi.Mu Gloves. Mi.mu gloves official website. <https://mimugloves.com/about/>, 2026. Accessed: 2026-05-07.
- [7] PyAudio Developers. Pyaudio. <https://pypi.org/project/PyAudio/>, 2023. Python audio playback library. Accessed: 2026-05-07.